Telcoin Solana Peg Stability Vault

Security Assessment

May 28, 2026

Prepared for:
Chase Brown
Telcoin

Prepared by: **Kevin Valerio**

Table of Contents

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Kevin Valerio, Consultant
kevin.valerio@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
April 30, 2026	Pre-project kickoff call
May 8, 2026	Delivery of report draft
May 11, 2026	Report readout meeting
May 13, 2026	Review of issues fixes
May 28, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

Telcoin engaged Trail of Bits to review the security of Solana's Peg Stability Vault (PSV).

Solana PSV is an Anchor-based Solana program that implements a Peg Stability Vault for swapping between Telcoin's stablecoin and another stablecoin (gem token), such as USDC. Its business goal is to help maintain a 1:1 peg by letting users or arbitrageurs swap stablecoins to gem (e.g., Telcoin's eUSD to USDC), with optional input/output fees, slippage protection, allowlist-based fee exemptions, and treasury-controlled reserve withdrawals. The main on-chain state is a Vault PDA derived from the stable and gem mints, which owns the vault token accounts for both assets. The program supports classic SPL Token and Token-2022. The admin can take fees on swaps, control rate limits, transfer roles, manage the whitelist, pause and unpause, withdraw funds, and close the vault.

A team of one consultant conducted the review from May 4 to May 8, 2026, for a total of one engineer-week of effort. Our testing efforts focused on reviewing every feature of the program; this included not only the stablecoin swapping mechanism, but also admin-only features such as allowlisting, rate-limiting, and fee control. With full access to source code, documentation and the EVM implementation of the stablecoin swapping, we performed static and dynamic testing of the Solana PSV, using automated and manual processes. Scripts were considered to be out of the scope of this audit.

Observations and Impact

The Solana PSV codebase is narrow and easy to reason about. Its fixed 1:1 pricing reduces the protocol's exposure to price-manipulation attacks. The review did not identify severe vulnerabilities under the stated trust assumptions. However, when integrating a Token-2022 stablecoin, we recommend deeply reviewing its mint extensions, since these could change the vault's properties and security assumptions, as illustrated in TOB-TELCO-04.

Recommendations

Based on the codebase maturity evaluation and findings identified during the security review, Trail of Bits recommends that Telcoin take the following steps:

- **Remediate any findings**. These should be addressed through direct fixes or by hardening the affected code.
- **Define a strict token onboarding process**. Before a stable or gem token is added to a PSV vault, confirm the token program; review all Token-2022 extensions when present; and document transfer fees, transfer hooks, freeze behavior,

confidential-transfer behavior, and any authority that can change token behavior after deployment.

- **Strengthen deployment controls**. Deployment scripts should verify the vault address, admin, treasury, token mints, token program IDs, vault token accounts, and token-account owners before any vaults are funded with stablecoins.
- **Expand verification with continuous integration (CI)**. Add CI that runs the Rust and Anchor test suites on every pull request, as this will help ensure that agentic development does not accidentally introduce unwanted behaviors.
- **Introduce invariants.** Add invariants to the program. Use a Solana fuzzer such as Trident to stress the program, potentially finding a sequence of instructions that would break the invariants, and include that in your CI. Appendix D explains how invariants are defined (under `#flow` macro) using Trident.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	0
Medium	2
Low	0
Informational	3
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Access Controls	1
Data Validation	3
Error Reporting	1

Project Goals

The engagement was scoped to provide a security assessment of the Telcoin Solana Peg Stability Vault. Specifically, we sought to answer the following non-exhaustive list of questions:

- Can `buy_gem` and `sell_gem` be manipulated to break 1:1 swap accounting, drain reserves, bypass slippage checks, or extract value through fee rounding?
- Are the vault PDA, vault ATAs, mint checks, and token program constraints sufficient?
- Does the Token-2022 and SPL Token support handle relevant mint extensions without unexpected drain, denial of service (DoS), or censorship risk?
- Can vault initialization be front-run, misconfigured, or permanently blocked?
- Are the admin, treasury, pauser, and unpauser roles protected against unauthorized use?
- Can the rate-limit mechanism be bypassed, reset, or used to deny service across buy and sell directions within the same slot?
- Can allowlist PDAs be forged, reused across vaults, or used to bypass fees for an unintended user?
- Are collected fees sent only to the treasury-owned token accounts?
- Can unprivileged accounts abuse the `pause`, `unpause`, and `close_vault` flows?
- Does the Solana port preserve the security properties of the Solidity PSV?
- Can the codebase handle arithmetic, rounding, dust amounts, and large swap amounts without causing overflows or unexpected behavior?

Project Targets

The engagement involved reviewing and testing the following target.

Solana Peg Stability Vault

Repository	https://github.com/telcoin/solana-psv
Version	`3443e9bf6c7ee0da9a90ad78d0e4a222125155c3`
Type	Rust
Platform	Solana

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- We manually reviewed the Solana PSV program under `programs/solana-psv/src/lib.rs`, including vault initialization, admin changes, treasury changes, pausing, allowlist management, fee configuration, rate limits, preview functions, swaps, and withdrawals.
- We used the Solidity implementation in the `tdab-stablecoin` repository as a contextual reference for the Solana port.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- We did not review build configuration, CI configuration, or deployment configuration.
- We did not review deployment scripts, including the files under `scripts` directory.

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project:

Tool	Description	Policy
Trident	A Rust-based stateful fuzzer for Solana programs. It executes randomized instruction flows against the compiled SBF program and checks invariants after each flow.	Appendix D
Internal AI bug hunting agent	An in-house AI-driven tool that performs automated vulnerability discovery by analyzing program logic, identifying candidate vulnerability patterns, and generating hypotheses for manual verification.	

Areas of Focus

Our automated testing and verification work focused on the following system properties:

- Swap accounting and one-to-one ratio conservation
- Allowlist, pause, and rate-limit behavior

Test Results

The results of this focused testing are detailed below.

Solana PSV swap and control-flow invariants
We used Trident to initialize a mixed Token-2022/SPL Token PSV vault, then execute randomized buy, sell, allowlist, pause, and rate-limit flows. The harness checks state before and after each flow so both successful and failed transactions are covered.

The following table lists the invariants we tested.

Property	Tool	Result
A successful swap must not create or destroy stables or gems.	Trident	**Passed**
A failed swap must leave balances and vault state unchanged.	Trident	**Passed**
A successful buy must move stable in and the same gross amount of gem out.	Trident	**Passed**
A successful sell must move gem in and the same gross amount of stable out.	Trident	**Passed**
A swap must not change roles, mints, fees, limits, whitelist count, or bump.	Trident	**Passed**
A random whitelist account must not bypass fees.	Trident	**Passed**
A valid whitelist account must pay no fees.	Trident	**Passed**
A non-admin must not change fees.	Trident	**Passed**
A buy while paused must fail and change nothing.	Trident	**Passed**
A sell while paused must fail and change nothing.	Trident	**Passed**
A zero-fee buy then sell of the same amount must end with the same balances.	Trident	**Passed**
A swap over the slot limit must fail.	Trident	**Passed**
A swap blocked by the slot limit must change nothing.	Trident	**Passed**

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development life cycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	The arithmetic used is limited and easy to trace. Fees use basis points with constants, and the core formula is isolated in `calculate_swap_output`. The implementation uses `u128` intermediate values, checked arithmetic, and ceiling division, while the release profile enables overflow checks. The formula is also documented in the README.	**Satisfactory**
Auditing	The program defines event types for swaps, fee updates, rate-limit updates, role changes, allowlist changes, pause state changes, withdrawals, and previews. Most state-changing handlers emit events after state update or transfer. The initialization flow only logs initialization event instead of emitting them, which makes vault creation less consistent with the rest of the program.	**Moderate**
Authentication / Access Controls	The program separates privileges across admin, treasury, pauser, and unpauser roles. Anchor constraints enforce the signer for privileged instructions, and the admin and treasury transfers use a two-step transfer flow, so the new privileged account must explicitly accept the role. The access-control model is also tested across role transfer, unauthorized calls, pause and unpause behavior, withdrawals, and close-vault paths. A remaining improvement is that `set_pauser` and `set_unpauser` reject the zero address but do not reject the vault PDA, while admin and treasury role changes include that check.	**Satisfactory**
Complexity Management	The codebase keeps the implementation in one `lib.rs` file and uses clear names for state, instructions, account	**Satisfactory**

	contexts, helper functions, and errors. The core helper functions isolate fee calculation and rate-limit handling, and Anchor constraints contain the proper required validations. However, the swap handlers contain repeated transfer and signer-seed logic.	
Cryptography and Key Management	The program does not implement custom cryptography, randomness, or nonce management. It relies on Solana and Anchor primitives for signer checks, PDA derivation, and CPI signer seeds, so it is not applicable.	**Not Applicable**
Decentralization	The admin can change fees, rate limits, pauser, unpauser, allowlist entries, and pending role transfers. The treasury can withdraw reserves, and close-vault requires both admin and treasury relationships through account constraints. However, the repository does not provide enough context to rate decentralization maturity. Key custody, multisig usage, upgrade authority, and timelocks require confirmation outside the codebase.	**Further Investigation Required**
Documentation	The documentation gives a clear overview of the PSV, the supported token programs, the initialization assumptions, the role model, the fee formula, rate limits, test commands, deployment scripts, account structure, and the Solana-specific background. Inline comments and Rust doc comments also explain important account constraints and instruction behavior. The documentation is strong enough for a satisfactory rating because a developer can understand the protocol flow without reading every line of implementation first.	**Satisfactory**
Low-Level Manipulation	The program does not use unsafe Rust, inline assembly, or raw low-level Solana instruction calls. Token operations are performed through `anchor_spl::token_interface`, with `transfer_checked` and `close_account` CPIs.	**Not Applicable**
Testing and Verification	The tests use LiteSVM and load the compiled SBF program, while the TypeScript tests use Anchor's local	**Satisfactory**

	validator. The tests cover normal paths, failure paths, and many edge cases for initialization, swaps, access controls, role transfers, pause behavior, allowlist behavior, rate limits, withdrawals, close-vault behavior, slippage, fee bounds, arithmetic rounding, optional treasury accounts, decimal mismatch, and dust sweeping. We recommend adding invariants for fuzzing campaigns, as this would be the natural next step.	
Transaction Ordering	The swap design has limited transaction-ordering exposure because pricing is fixed at 1:1, the program does not read an oracle or pool spot price, and user swaps include `min_amount_out` checks. Transaction ordering is not relevant.	**Not Applicable**

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Permissionless vault initialization can capture vault control	Access Controls	**Medium**
2	Preview quotes ignore rate limits	Error Reporting	**Informational**
3	Per-transaction rate limit can be bypassed with multiple swap instructions	Data Validation	**Informational**
4	Missing Token-2022 transfer fee validation can drain vault reserves	Data Validation	**Informational**
5	Closed vault PDA can be reinitialized by an attacker	Data Validation	**Medium**

Detailed Findings

1. Permissionless vault initialization can capture vault control	
Severity: **Medium**	Difficulty: **Medium**
Type: Access Controls	Finding ID: TOB-TELCO-01
Target: `programs/solana-psv/src/lib.rs`	

Description
The `initialize` instruction is permissionless. This allows the first caller for a token pair to control the vault for that pair straight after program deployment.

The vault address is a PDA derived only from the stable mint and gem mint. The account constraints create the vault with `init` and use the caller-provided `admin` account as payer.

The instruction does not check that the caller is an approved deployer. If funds are later sent to the vault token accounts, the attacker can use the stored treasury role to withdraw them.

Exploit Scenario
Eve observes Telcoin's team deploying a PSV Vault for eUSD/USDC. She reads the program's code, and believes that the team will soon deploy the same for eUSD/USDT. She obtains the eUSD and USDT mint addresses before the project initializes the PSV Vault and derives the vault PDA from them. Later, the team deploys the vault but does not yet call `initialize`. Eve calls `initialize` immediately following deployment with `admin` and `treasury` as accounts she controls, thereby assigning herself those privileged roles on the deployed program.

Recommendations
Short term, require `admin` to match an approved deployer in `initialize`. Alternatively, require the signer of the `initialize` function to match `upgrade_authority_address`, which is set at deployment time. See the example in Wormhole's Wold-ID program.

Long term, bind all vault identities to the Telcoin's team known authority. Require the deployment scripts to verify the on-chain `admin`, `treasury`, `stable_mint`, `gem_mint`, and vault token owners before funding any vault. Do not send funds before you ensure that the program's admin and treasury addresses are fully controlled by the team.

References

- How do you avoid frontrunning when initializing a program with a PDA? - Solana Stack Exchange
- How to intialize programs with default values? - Solana Stack Exchange

2. Preview quotes ignore rate limits

Severity: **Informational**	Difficulty: **Low**
Type: Error Reporting	Finding ID: TOB-TELCO-02
Target: `programs/solana-psv/src/lib.rs`	

Description

The preview instructions for buying and selling paths emit a quote even when the swap would fail due to rate limits. This makes the preview output unreliable for frontends, and it can cause users to send failing transactions and pay fees for transactions that never execute.

In contrast, the swap instructions enforce rate limits by calling `handle_rate_limits` from `buy_gem` and `sell_gem`. As a result, a frontend can obtain a quote for an input amount that the program rejects with `ExceedsTransactionLimit` or `ExceedsSlotLimit` during execution.

```
pub fn preview_sell_gem(ctx: Context<PreviewSwap>, amount_in: u64) -> Result<()> {
    // Preview should reflect actual swap behavior including pause state
    require!(!ctx.accounts.vault.paused, PsvError::Paused);
    require!(amount_in > 0, PsvError::ZeroAmount);
    let is_whitelisted = ctx.accounts.whitelist.is_some();

    // Calculate output amount and fee using tout (output fee for selling)
    // If whitelisted, fee will be 0 regardless of tout value
    let (amount_out, fee) =
        calculate_swap_output(amount_in, ctx.accounts.vault.tout, is_whitelisted)?;
```

Figure 2.1: `preview_sell_gem` does not consider rate limiting and directly calculates swap output (solana-psv/programs/solana-psv/src/lib.rs#L1019–L1028)

Exploit Scenario

Alice uses a frontend that calls `preview_buy_gem` to display the expected output for an input of 1,000,000 eUSD. The vault configuration sets `max_per_transaction` to 100,000 units, so the trade will not be executed. The preview call still emits a quote; Alice submits a swap based on that quote, and the transaction fails due to the rate limit check. Alice pays fees for a transaction that cannot succeed.

Recommendations

Short term, validate rate limits in `preview_buy_gem` and `preview_sell_gem`, and return the same errors as swaps when limits are exceeded.

Long term, add tests that enable rate limits and assert that preview instructions reject inputs that exceed those limits.

3. Per-transaction rate limit can be bypassed with multiple swap instructions

Severity: **Informational**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-TELCO-03
Target: `programs/solana-psv/src/lib.rs`	

Description
The per-transaction rate limit is enforced per swap instruction instead of per Solana transaction. A user can include several swap instructions in the same transaction, and each instruction checks only its own `amount_in`. This lets the transaction move more tokens than the configured `max_per_transaction` value.

This creates a mismatch between the limit name and the implemented behavior. The control can still limit the size of a single swap instruction, but it does not limit the total amount swapped by one Solana transaction.

Exploit Scenario
Eve observes that the eUSD/USDC vault has `max_per_transaction` set to 1,000 and `max_per_slot` disabled. Eve creates one Solana transaction with two 1,000 eUSD `buy_gem` instructions. This passes the rate-limit check, and the transaction swaps 2,000 eUSD.

Recommendations
Short term, either rename the field (and update the docs) to clarify that the limit applies per swap instruction, or enforce `max_per_transaction` against the cumulative swap amount in the transaction.

Long term, add rate-limit tests that include multiple PSV instructions in one Solana transaction for `buy_gem`, `sell_gem`, and mixed buy/sell flows.

4. Missing Token-2022 transfer fee validation can drain vault reserves

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-TELCO-04
Target: `programs/solana-psv/src/lib.rs`	

Description
When a vault uses a mint with a transfer fee (`TransferFeeConfig`) for a Token-2022 stablecoin, it considers the brutto amount instead of the net amount.

If the stable mint charges a transfer fee, the vault receives less stable than `amount_in`, but PSV still pays as if the full amount arrived. The same pattern exists in `sell_gem`.

As an example, PYUSD and USDG on Solana both have a `TransferFeeConfig` enabled, but with 0 basis points and maximum fee 0. While this still makes current swaps safe, if one of these tokens later sets a nonzero transfer fee, PSV will continue to pay users from the nominal swap amount and will lose reserves on each affected swap. We haven't found any stablecoin on Solana as of today with a `transferFeeBasisPoints` > 0, therefore the difficulty is set as high and the severity as informational.

Exploit Scenario
Alice uses a PSV vault configured to use eUSD/PYUSD. Everything works properly, until one day PayPal's PYUSD changes the `TransferFeeConfig` to have one basis point fee for each transfer. Alice calls `buy_gem` against 1000 PYUSD, and the PSV computes the swap from `amount_in = 1000` and sends Alice 1000 eUSD. However, because of the transfer fee, the vault receives 999 PYUSD. The vault drains little by little while the usable stable reserve does not increase.

Recommendations
Short term, reject Token-2022 mints with non-zero `TransferFeeConfig` during initialization, if the team does not plan to integrate this kind of stablecoin. Eventually, at swap-time, ensure that the targeted stablecoin does not have any fees, and throw an error if it does.

Long term, before adding a stablecoin during vault initialization, fully investigate the stablecoin's extensions, including `TransferFeeConfig`. Document which Token-2022 extensions you accept and which you reject.

References

- Solana Token Extensions transfer fee documentation

5. Closed vault PDA can be reinitialized by an attacker	
Severity: **Medium**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-TELCO-05
Target: `programs/solana-psv/src/lib.rs`	

Description
After a vault is closed, any signer can reinitialize the same PDA with attacker-controlled roles. The `Initialize` context creates the vault PDA with `init` using only the stable mint and gem mint as seeds.

The `close_vault` instruction closes the vault account after the admin pauses it and removes the allowlist. The handler sweeps token balances and closes the vault token accounts, so funds in the old vault are safe.

The risk starts after closure. The same PDA can hold a new attacker state. Any frontend, indexer, or bot that caches the vault address keeps using it as the trusted vault. Users on stale integrations swap against the attacker vault and pay fees to the attacker treasury.

Exploit Scenario
Alice closes the eUSD/USDC vault during a migration. Eve, an attacker, derives the same PDA from the eUSD and USDC mints and calls `initialize` with herself as admin, treasury, pauser, and unpauser. Eve sets `tin` and `tout` to 1,000 basis points and deposits enough reserves for small swaps. An arbitrageur, running a bot to support the 1:1 pegging, still has the old PDA as the eUSD/USDC PSV. It swaps 1,000 eUSD, and the program sends 900 USDC-equivalent to Bob and 100 USDC-equivalent to Eve's treasury.

Recommendations
Short term, restrict `initialize` so vault roles cannot be assigned to arbitrary callers. Fixing TOB-TELCO-01 also fixes this, as an attacker could not call `initialize` on an already-deployed program, since the attacker would not have been set as the upgrade authority.

Long term, track vault lifecycle to monitor initialized and closed token pairs.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	**Description**
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Low-Level Manipulation	The justified use of inline assembly and low-level calls
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	**Description**
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Code Quality Recommendations

This appendix contains recommendations for findings that do not have immediate or obvious security implications. However, implementing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Emit an event when a vault is initialized.** Most operations emit events, but `initialize` writes only `msg!("PSV Vault initialized")`. Add an initialization event with the vault, stable mint, gem mint, admin, treasury, pauser, unpauser, `tin`, and `tout` so indexers can track vault creation.
- **Closing the vault should check accounts more strictly**. In `close_vault`, the context does not enforce that accounts match `vault.stable_mint` and `vault.gem_mint` like the swap instructions do. If someone passes the wrong accounts, the instruction will fail later, but these checks should be constraints so it fails early.
- **Reject the vault PDA for pauser roles.** `set_pauser` and `set_unpauser` reject the zero pubkey, but they can still store the vault PDA as pauser or unpauser. That PDA cannot sign `pause` or `unpause`. Add the same `!= vault.key()` guard used for treasury and admin role updates.
- **Rename slippage wording to minimum amount**. In DeFi, the term "slippage" is used when liquidity affects the trade outcome. Since the application always uses a 1:1 ratio, "slippage" is confusing because no slippage can happen. Consider using another term (e.g., "minimum amount") instead.
- **Rename the directional fee parameters**. Replace `tin` and `tout` with names like `stable_to_gem_fee_bps` and `gem_to_stable_fee_bps`, because this code reverses the Maker PSM terminology. In Maker PSM, `tin` is the fee when users sell gem, and `tout` is the fee when users buy gem, so Maker's code uses `tin` in `sellGem` and `tout` in `buyGem`. The Solana program does the opposite.
- **Update dependencies.** While not directly exploitable, `cargo-audit` found seven vulnerable dependencies that should be updated. Important crates such as `anchor-lang` and `anchor-spl` are set to version `0.32.1`, while the latest version is `1.0.2`. These should be updated.

D. Invariant-based Fuzzing with Trident

This appendix shows how Trident invariants should be written for PSV. They should describe the properties and not copy the program's logic; the fuzzer will then try to break those properties.

A useful Trident invariant has three parts. First, set up the smallest state needed for the property. Second, take a snapshot of the accounts that matter. Third, execute one or more instructions and compare the post-state to the property. Below is an example of an invariant defined as a `#[flow]`.

```
#[flow]
fn zero_fee_roundtrip_does_not_create_value(&mut self) {
    let world = self.world();
    self.ensure_unpaused(world);
    self.set_fees(world, 0, 0);
    self.set_rate_limits(world, 0, 0);

    let amount = self.trident.random_from_range(1..=50_000_u64);
    self.top_up_liquidity(world, amount);
    let before = self.snapshot(world);
    let buy_ix = ix_buy_gem(world, amount, amount, psv_program_id());
    let sell_ix = ix_sell_gem(world, amount, amount, psv_program_id());
    let res = self
        .trident
        .process_transaction(&[buy_ix, sell_ix], Some("zero_fee_roundtrip"));
    let after = self.snapshot(world);

    assert!(res.is_success(), "zero-fee buy+sell roundtrip failed: {:#?}",
        res.get_result()
    );
    assert_eq!(after.balances, before.balances,"zero-fee roundtrip changed balances");
    assert_swap_config_unchanged(&before.vault, &after.vault);
}
```

Figure D.1: Invariant ensuring that null fees make balances even

E. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On May 13, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Telcoin team for the issues identified in this report. Each fix was evaluated to determine its effectiveness in resolving the associated issue. Telcoin provided the fixes in a new commit: `dbebbda1de0afddbc0e444a04a149b667ce64f4e`.

Trail of Bits also noted a few follow-up items while reviewing the fixes. These do not introduce security vulnerabilities, but Telcoin should address or account for them:

- Update the `README`, since it still references the old Anchor versions.
- Add the same `programData` derivation and account from `tests/solana-psv.ts` to `scripts/initialize-devnet.ts`. This is properly done in `solana-psv.ts` for testing but not in the devnet script. `accountPartials` should also include the program data, as this is needed to link the program authority (for `initialize`). Otherwise, if this script is used in the future, it will fail to run.
- One of Anchor's 1.0 new features is "duplicate mutable accounts disallowed by default." In PSV's context, this means if the treasury performs swaps, it will fail if fees are non-zero (because PSV passes that same ATA twice as mutable). While this does not pose any security risk, the team should be aware of this behavior.
- If an admin transfer is done and the new admin closes the vault, the original program upgrade authority can still re-initialize the vault while the new admin cannot unless he also holds the program upgrade authority (this is possible with `solana program set-upgrade-authority`). As a result, the team should consider whether or not to transfer the upgrading authority to the new admin when an admin-role transfer is completed.

In summary, Telcoin has resolved all five issues described in this report. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Permissionless vault initialization can capture vault control	Medium	Resolved
2	Preview quotes ignore rate limits	Informational	Resolved
3	Per-transaction rate limit can be bypassed with multiple swap instructions	Informational	Resolved
4	Missing Token-2022 transfer fee validation can drain vault reserves	Informational	Resolved
5	Closed vault PDA can be reinitialized by an attacker	Medium	Resolved

Detailed Fix Review Results

TOB-TELCO-01: Permissionless vault initialization can capture vault control
Resolved in commit `dbebbda1`. The initialize instruction is no longer permissionless; it now requires the admin signer to match the program's upgrade authority by checking `upgrade_authority_address == Some(admin)`. This prevents a third party from front-running the initialization.

TOB-TELCO-02: Preview quotes ignore rate limits
Resolved in commit `dbebbda1`. `preview_buy_gem` and `preview_sell_gem` now validate rate limits via `check_rate_limits` (via a similar logic as `handle_rate_limits`), and they return the same errors as real swaps (`ExceedsSwapLimit` / `ExceedsSlotLimit`).

TOB-TELCO-03: Per-transaction rate limit can be bypassed with multiple swap instructions
Resolved in commit `dbebbda1`. The team treated this as a naming mismatch; `max_per_transaction` was renamed to `max_per_swap` and the error to `ExceedsSwapLimit` to match the real behavior.

TOB-TELCO-04: Missing Token-2022 transfer fee validation can drain vault reserves
Resolved in commit `dbebbda1`. The fix adds a `TransferFeeConfig` check that rejects Token-2022 mints with any non-zero transfer fee during initialize and again at `buy_gem`/`sell_gem` time, so that the vault never operates with a stablecoin that has fees on transfer (even if fees are enabled later). A test suite was added to ensure that non-zero fees are rejected while an all-zero `TransferFeeConfig` is allowed.

TOB-TELCO-05: Closed vault PDA can be reinitialized by an attacker
Resolved in commit `dbebbda1`. This issue was remediated in the same manner as TOB-TELCO-01: `initialize` is now restricted to the program upgrade authority, so after a vault is closed, an attacker cannot recreate the same `(stable_mint, gem_mint)` PDA with chosen admin/treasury/pauser roles.

F. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.